PDFTriage: Question Answering over Long, Structured Documents

Jon Saad-Falcon
Stanford University
jonsaadfalcon@stanford.edu

Joe Barrow
Adobe Research
jbarrow@adobe.com

Alexa Siu
Adobe Research
asiu@adobe.com

Ani Nenkova
Adobe Research
nenkova@adobe.com

David Seunghyun Yoon
Adobe Research
syoon@adobe.com

Ryan A. Rossi
Adobe Research
ryrossi@adobe.com

Franck Dernoncourt
Adobe Research
dernonco@adobe.com

arXiv:2309.08872v2 [cs.CL] 8 Nov 2023

Abstract

Large Language Models (LLMs) have issues with document question answering (QA) in situations where the document is unable to fit in the small context length of an LLM. To overcome this issue, most existing works focus on retrieving the relevant context from the document, representing them as plain text. However, documents such as PDFs, web pages, and presentations are naturally structured with different pages, tables, sections, and so on. Representing such structured documents as plain text is incongruous with the user's mental model of these documents with rich structure. When a system has to query the document for context, this incongruity is brought to the fore, and seemingly trivial questions can trip up the QA system. To bridge this fundamental gap in handling structured documents, we propose an approach called *PDFTriage* that enables models to retrieve the context based on either structure or content. Our experiments demonstrate the effectiveness of the proposed *PDFTriage-augmented* models across several classes of questions where existing retrieval-augmented LLMs fail. To facilitate further research on this fundamental problem, we release our benchmark dataset consisting of 900+ human-generated questions over 80 structured documents from 10 different categories of question types for document QA. Our code and datasets will be released soon on Github.

1 Introduction

When a document does not fit in the limited context window of an LLM, different strategies can be deployed to fetch relevant context. Current approaches often rely on a pre-retrieval step to fetch the relevant context from documents (Pereira et al., 2023; Gao et al., 2022). These pre-retrieval steps tend to represent the document as plain text chunks, sharing some similarity with the user query and potentially containing the answer. However, many document types have rich structure, such as web pages, PDFs, presentations, and so on. For these structured documents, representing the document as plain text is often incongruous with the user's mental model of a *structured document*. This can lead to questions that, to users, may be trivially answerable, but fail with common/current approaches to document QA using LLMs. For instance, consider the following two questions:

Q1 "Can you summarize the key takeaways from pages 5-7?"

Q2 "What year *[in table 3]* has the maximum revenue?"

In the first question, document structure is *explicitly referenced* ("pages 5-7"). In the second question, document structure is *implicitly referenced* ("*in table 3*"). In both cases, a representation of document structure is necessary to identify the salient context and answer the question. Considering the document as plain text discards the relevant structure needed to answer these questions.

We propose addressing this simplification of documents by allowing models to retrieve the context based on either structure or content. Our approach, which we refer to as *PDFTriage*, gives models access to metadata about the structure of the document. We leverage document structure by augmenting prompts with both document structure metadata and a set of model-callable retrieval functions over various types of structure. For example, we introduce the `fetch_pages(pages: list[int])` function, which allows the model to fetch a list of pages. We show that by providing the structure and the ability to issue queries over that structure, PDFTriage-augmented models can reliably answer several classes of questions that plain retrieval-augmented LLMs could not.

In order to evaluate our approach, we construct a dataset of roughly 900 human-written questions

over 90 documents, representing 10 different categories of questions that users might ask. Those categories include "document structure questions", "table reasoning questions", and "trick questions", among several others. We will release the dataset of questions, documents, model answers, and annotator preferences. In addition, we release the code and prompts used.

The key contributions of this paper are:

- We identify a gap in question answering over structured documents with current LLM approaches, namely treating documents as plain text rather than structured objects;
- We release a dataset of tagged question types, along with model responses, in order to facilitate further research on this topic; and
- We present a method of prompting the model, called *PDFTriage*, that improves the ability of an LLM to respond to questions over structured documents.

The rest of the paper proceeds as follows: in Section 2, we identify the related works to this one, and identify the distinguishing features of our work; in Section 3 we outline the *PDFTriage* approach, including the document representation, the new retrieval functions, and the prompting techniques; in Section 4 we outline how we constructed the evaluation dataset of human-written questions; in Section 5 we detail the experiments we run to support the above contributions; in Section 6 we list the key takeaways of those experiments; and, lastly, in Section 7 we describe the limitations of our current work and future directions.

2 Related Works

2.1 Tool and Retrieval Augmented LLMs

Tool-augmented LLMs have become increasingly popular as a way to enhance existing LLMs to utilize tools for responding to human instructions (Schick et al., 2023). ReAct (Yao et al., 2022) is a few-shot prompting approach that leverages the Wikipedia API to generate a sequence of API calls to solve a specific task. Such task-solving trajectories are shown to be more interpretable compared to baselines. Self-ask (Press et al., 2022) prompt provides the follow-up question explicitly before answering it, and for ease of parsing uses a specific scaffold such as "Follow-up question:" or "So the final answer is:". Toolformer (Schick et al., 2023) uses self-supervision to teach itself to use tools by leveraging the few-shot capabilities of an LM to obtain a sample of potential tool uses, which is then fine-tuned on a sample of its own generations based on those that improve the model's ability to predict future tokens. TALM (Parisi et al., 2022) augments LMs with non-differentiable tools using only text along with an iterative technique to bootstrap performance using only a few examples. Recently, Taskmatrix (Liang et al., 2023) and Gorilla (Patil et al., 2023) have focused on improving the ability of LLMs to handle millions of tools from a variety of applications. There have also been many works focused on benchmarks for tool-augmented LLMs (Li et al., 2023; Zhuang et al., 2023). These include API-Bank (Li et al., 2023), focused on evaluating LLMs' ability to plan, retrieve, and correctly execute step-by-step API calls for carrying out various tasks, and ToolQA (Zhuang et al., 2023) that focused on question-answering using external tools.

Retrieval-augmented language models aim to enhance the reasoning capabilities of LLMs using external knowledge sources for retrieving related documents (Asai et al., 2022; Gao et al., 2022; Lin et al., 2023; Yu et al., 2023; Zhao et al., 2023; Feng et al., 2023). In particular, HyDE (Gao et al., 2022) generates a hypothetical document (capturing relevance patterns) by zero-shot instructing an instruction-following LLM, then encodes the document into an embedding vector via an unsupervised contrastively learned encoder, which is used to retrieve real documents that are similar to the generated document. More recently, Feng et al. (2023) proposed InteR that iteratively refines the inputs of search engines and LLMs for more accurate retrieval. In particular, InteR uses search engines to enhance the knowledge in queries using LLM-generated knowledge collections whereas LLMs improve prompt formulation by leveraging the retrieved documents from the search engine. For further details on augmented language models, see the recent survey (Mialon et al., 2023).

2.2 Question Answering

Much of the existing work in QA does not ground the questions in structured documents, instead primarily focusing on extractive QA tasks such as GLUE (Wang et al., 2018). For example, text-only documents in QA datasets, like SQuAD (Rajpurkar et al., 2016) and NaturalQuestions (Kwiatkowski

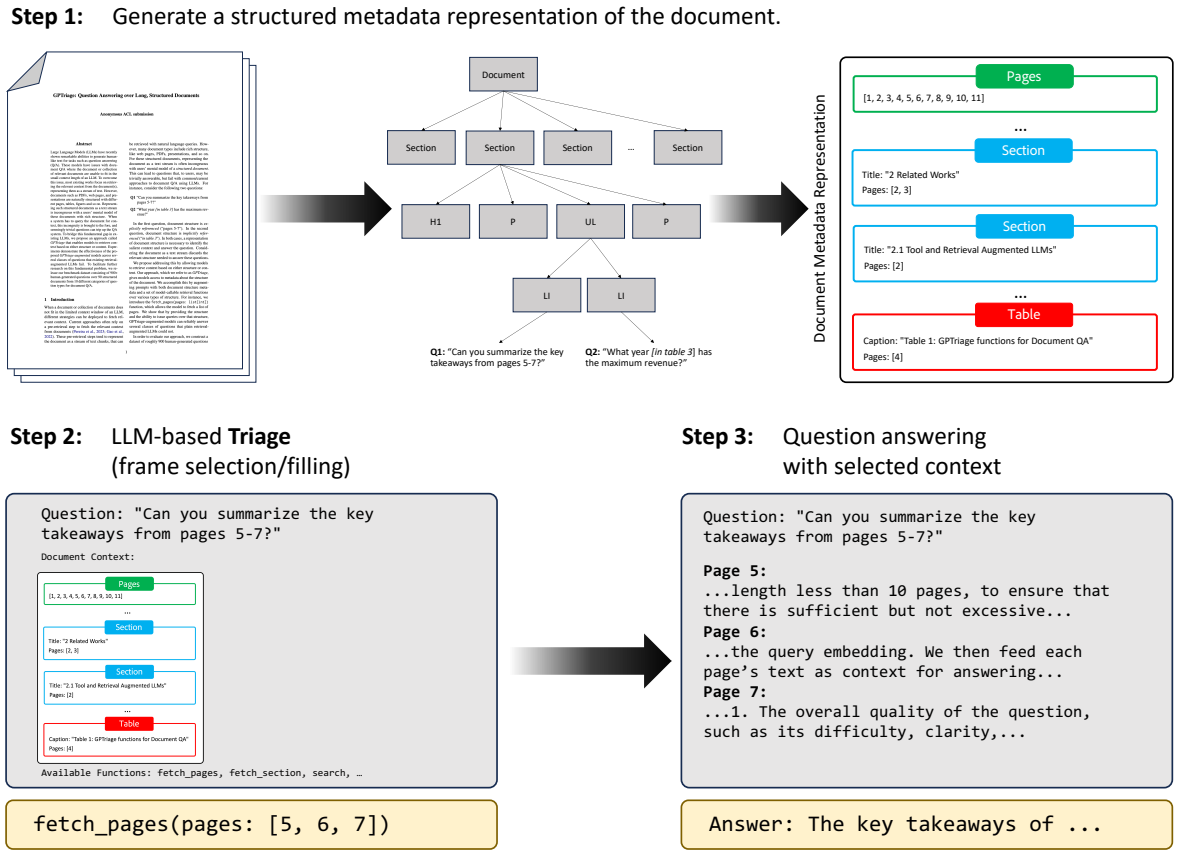

Figure 1: **Overview of the PDFTriage technique**: PDFTriage leverages a PDF's structured metadata to implement a more precise and accurate document question-answering approach. It starts by generating a structured metadata representation of the document, extracting information surrounding section text, figure captions, headers, and tables. Next, given a query, a LLM-based Triage selects the document frame needed for answering the query and retrieves it directly from the selected page, section, figure, or table. Finally, the selected context and inputted query are processed by the LLM before the generated answer is outputted.

et al., 2019), don't contain tables or figures.

Document Question Answering . Several datasets have been constructed to benchmark different aspects of document-focused question-answering. DocVQA (Mathew et al., 2021) is a visual question-answering dataset focused that uses document scans. A recent work by Landeghem et al. (2023) focused on a dataset for document understanding and evaluation called DUDE, which uses both scans and born-digital PDFs. Both DUDE and DocVQA have questions that can be answered short-form; DUDE answers average roughly 3.35 tokens and DocVQA tokens average 2.11 tokens. QASPER (Dasigi et al., 2021) is a dataset focused on information-seeking questions and their answers from research papers, where the documents are parsed from raw LaTeXsources and the questions are primarily focused on document contents. The PDFTriage evaluation dataset seeks to expand on the question types in these datasets, getting questions that can reference the document structure or content, can be extractive or abstractive, and can require long-form answers or rewrites.

3 PDFTriage: Structured Retrieval from Document Metadata

The *PDFTriage* approach consists of three steps to answer a user's question, shown in Figure 1:

1. **Generate document metadata (Sec. 3.1):** Extract the structural elements of a document and convert them into readable metadata.
2. **LLM-based triage (Sec. 3.2):** Query the LLM to select the precise content (pages, sections, retrieved content) from the document.
3. **Answer using retrieved content (Sec. 3.3):** Based on the question and retrieved content, generate an answer.

# of Documents	82
# of Questions	908
Easy Questions	393
Medium Questions	144
Hard Questions	266
"Unsure" Questions	105

Table 1: Dataset statistics for the PDFTriage evaluation dataset.

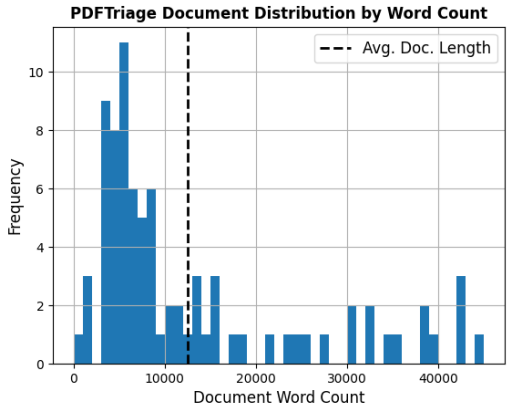

Figure 2: PDFTriage Document Distribution by Word Count

3.1 Document Representation

We consider *born-digital PDF documents* as the structured documents that users will be interacting with. Using the Adobe Extract API, we convert the PDFs into an HTML-like tree, which allows us to extract sections, section titles, page information, tables, and figures.[1] The Extract API generates a hierarchical tree of elements in the PDF, which includes section titles, tables, figures, paragraphs, and more. Each element contains metadata, such as its page and location. We can parse that tree to identify sections, section-levels, and headings, gather all the text on a certain page, or get the text around figures and tables. We map that structured information into a JSON type, that we use as the initial prompt for the LLM. The content is converted to markdown. An overview of this process is shown at the top of Figure 1.

3.2 LLM Querying of Document

PDFTriage utilizes five different functions in the approach: `fetch_pages`, `fetch_sections`, `fetch_table`, `fetch_figure`, and `retrieve`. As described in Table 2, each function allows the PDFTriage system to gather precise information related to the given PDF document, centering around structured textual data in headers, subheaders, figures, tables, and section paragraphs. The functions are used in separate queries by the PDFTriage system for each question, synthesizing multiple pieces of information to arrive at the final answer. The functions are provided and called in separate chat turns via the OpenAI function calling API,[2] though it would be possible to organize the prompting in a ReAct (Yao et al., 2022) or Toolformer (Schick et al., 2023) -like way.

3.3 Question Answering

To initialize PDFTriage for question-answering, we use the system prompt format of GPT-3.5 to input the following:

> You are an expert document question answering system. You answer questions by finding relevant content in the document and answering questions based on that content.
>
> Document: `<textual metadata of document>`

Using user prompting, we then input the query with no additional formatting. Next, the PDFTriage system uses the functions established in Section 2 to query the document for any necessary information to answer the question. In each turn, PDFTriage uses a singular function to gather the needed information before processing the retrieved context. In the final turn, the model outputs an answer to the question. For all of our experiments, we use the `gpt-35-turbo-0613` model.

4 Dataset Construction

To test the efficacy of PDFTriage, we constructed a document-focused set of question-answering tasks. Each task seeks to evaluate different aspects of document question-answering, analyzing reasoning across text, tables, and figures within a document. Additionally, we wanted to create questions ranging from single-step answering on an individual document page to multi-step reasoning across the whole document.

[1] https://developer.adobe.com/document-services/apis/pdf-extract/

[2] https://platform.openai.com/docs/api-reference

Function	Description
`fetch_pages`	Get the text contained in the pages listed.
`fetch_sections`	Get the text contained in the section listed.
`fetch_figure`	Get the text contained in the figure caption listed.
`fetch_table`	Get the text contained in the table caption listed.
`retrieve`	Issue a natural language query over the document, and fetch relevant chunks.

Table 2: PDFTriage Functions for Document QA.

We collected questions using Mechanical Turk.[3] The goal of our question collection task was to collect real-world document-oriented questions for various professional settings. For our documents, we sampled 1000 documents from the common crawl to get visually-rich, professional documents from various domains, then subsampled 100 documents based on their reading level (Flesch, 1948). [4] By collecting a broad set of document-oriented questions, we built a robust set of tasks across industries for testing the PDFTriage technique.

In order to collect a diverse set of questions, we generated our taxonomy of question types and then proceeded to collect a stratified sample across the types in the taxonomy. Each category highlights a different approach to document-oriented QA, covering multi-step reasoning that is not found in many other QA datasets. We asked annotators to read a document before writing a question. They were then tasked with writing a salient question in the specified category.

For our taxonomy, we consider ten different categories along with their associated descriptions:

1. **Figure Questions** (6.5%): Ask a question about a figure in the document.
2. **Text Questions** (26.2%): Ask a question about the document.
3. **Table Reasoning** (7.4%): Ask a question about a table in the document.
4. **Structure Questions** (3.7%): Ask a question about the structure of the document.
5. **Summarization** (16.4%): Ask for a summary of parts of the document or the full document.
6. **Extraction** (21.2%): Ask for specific content to be extracted from the document.
7. **Rewrite** (5.2%): Ask for a rewrite of some text in the document.
8. **Outside Questions** (8.6%): Ask a question that can't be answered with just the document.
9. **Cross-page Tasks** (1.1%): Ask a question that needs multiple parts of the document to answer.
10. **Classification** (3.7%): Ask about the type of the document.

In total, our dataset consists of 908 questions across 82 documents. On average a document contains 4,257 tokens of text, connected to headers, subheaders, section paragraphs, captions, and more. In Figure 2, we present the document distribution by word count. We provide detailed descriptions and examples of each of the classes in the appendix.

5 Experiments

We outline the models and strategies used in our approach along with our baselines for comparison. The code and datasets for reproducing our results will be released soon on Github.

5.1 PDFTriage

For our primary experiment, we use our PDFTriage approach to answer various questions in the selected PDF document dataset. This strategy leverages the structure of PDFs and the interactive system functions capability of GPT-3.5 to extract answers more precisely and accurately than existing naive approaches.

5.2 Retrieval Baselines

Page Retrieval . For our first baseline, we index the pages of each individual document using *text-embedding-ada-002* embeddings. Using cosine similarity, we retrieve the pages most similar to the query embedding. We then feed each page's text as context for answering the given question until we reach the context window limit for a model.

[3] https://mturk.com
[4] https://commoncrawl.org/

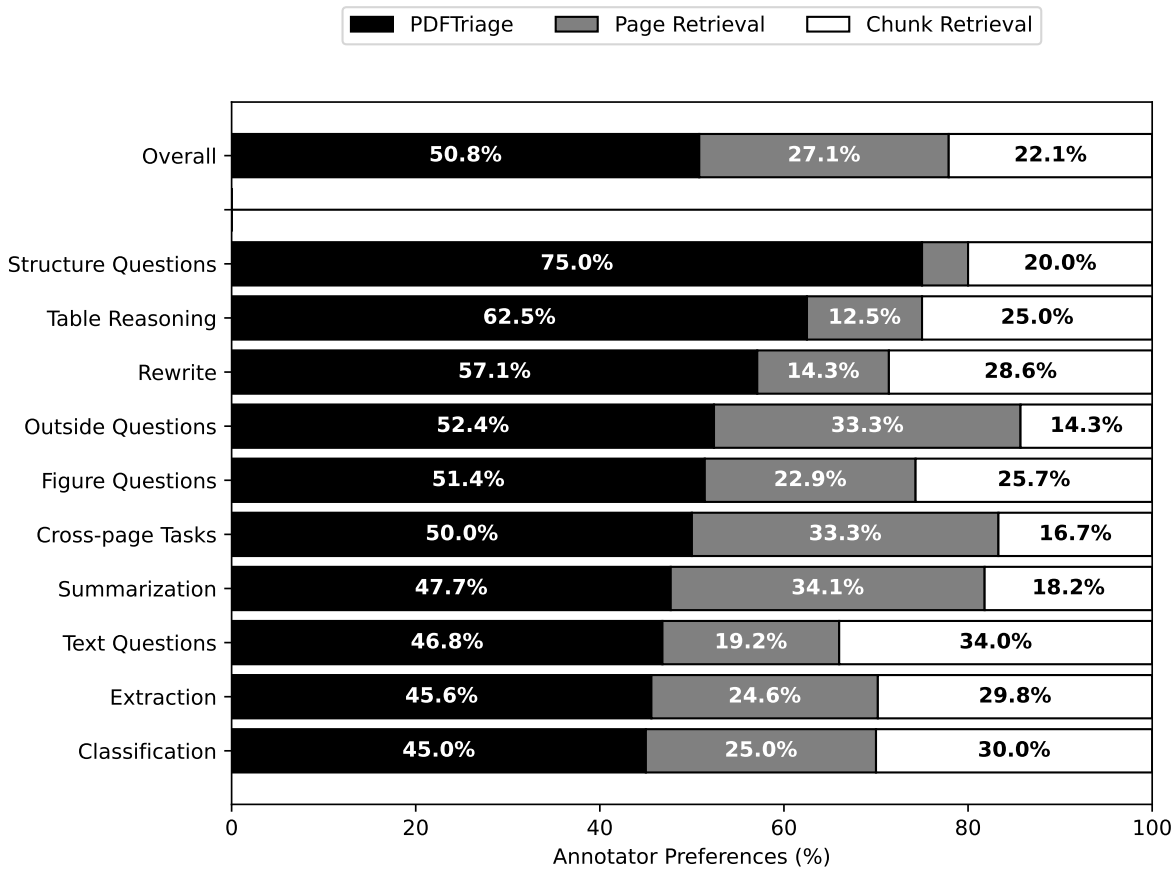

Figure 3: **User Preferences between PDFTriage and Alternate Approaches**: Overall, PDFTriage-generated answers were favored the most by the users, claiming 50.8% of the top-ranked answers overall. Furthermore, PDFTriage answers ranked higher on certain multi-page tasks, such as structure questions and table reasoning, while ranking lower on generalized textual tasks, such as classification and text questions. However, across all the question categories, PDFTriage beat both the Page Retrieval and Chunk Retrieval approaches on a head-to-head ranking.

Chunk Retrieval . In our second baseline, we concatenate all the document's text before chunking it into 100-word pieces. We then index each chunk using *text-embedding-ada-002* embeddings before using cosine similarity calculations to retrieve the chunks most similar to the query embedding. Finally, we feed each chunk's textual contents as context for answering the given question until we reach the context window limit for a model.

Prompting . For both retrieval baselines, we use the following prompt to get an answer from GPT-3.5:

> You are an expert document question answering system. You answer questions by finding relevant content in the document and answering questions based on that content.
>
> Document: `<retrieved pages/chunks>`
>
> Question: `<question>`

5.3 Human Evaluation

To measure any difference between PDFTriage and the retrieval baselines, we established a human labeling study on Upwork. In the study, we hired 12 experienced English-speaking annotators to judge the answers generated by each system. Please see Appendix A to see the full annotation questions for each question-document and its generated answers (for the overview, we use a sample question) as well as demographic information about the annotators.

Our questions seek to understand several key attributes of each question-document pair as well as the associated general questions:

1. The overall quality of the question, such as its difficulty, clarity, and information needed for answering it.

2. The category of the question, using the taxonomy in section 4.

3. The ranking of each generated answer for the given question-document pair.

4. The accuracy, informativeness, readability/understandability, and clarity of each generated answer.

6 Results and Analysis

In Table 1, we present the annotated question difficulty of each question in our sample. Overall, the largest group of questions (43.3%) were categorized as Easy while roughly a third of questions were categorized as Hard for various reasons.

In addition to question difficulty, we asked annotators to categorize questions by type using the same categories as Section 4. Our annotation framework results in a dataset that's diverse across both question types and question difficulties, covering textual sections, tables, figures, and headings as well as single-page and multi-page querying. The diversity of questions allows us to robustly evaluate multiple styles of document-centered QA, testing the efficacy of PDFTriage for different reasoning techniques.

6.1 PDFTriage yields better answers than retrieval-based approaches.

In our annotation study, we asked the annotators to rank PDFTriage compared to our two baselines, Page Retrieval and Chunk Retrieval (Section 5). In Figure 3, we found that annotators favored the PDFTriage answer over half of the time (50.7%) and favored the Chunk Retrieval approach over the Page Retrieval approach. When comparing different provided answers for the same question, PDFTriage performs substantially better than current alternatives, ranking higher than the alternate approaches across all the question types.

6.2 PDFTriage improves answer quality, accuracy, readability, and informativeness

In our annotation study, we also asked the annotators to score PDFTriage, Page Retrieval, and Chunk Retrieval answers across five major qualities: accuracy, informativeness, readability/understandability, and clarity. We hoped to better understand the strengths of each answer for users in document question-answering tasks. In Table 3, we show that PDFTriage answers score higher than Page Retrieval and Chunk Retrieval across all answer qualities except for Clarity. Crucially, PDFTriage had the highest scores for Overall Quality and Answer Accuracy. For annotator agreement, we calculated an average Cohen's kappa score of 0.584.

	PDFTriage	*Page Retrieval*	*Chunk Retrieval*
Readability	**4.2**	4.1	4.1
Informativeness	**3.9**	3.7	3.4
Clarity	2.0	2.1	**2.3**
Accuracy	**3.8**	3.6	3.4
Overall Quality	**3.9**	3.8	3.6

Table 3: Answer Quality Scoring

In Appendix A, we provide a high-resolution breakdown of annotations for "Overall Quality" and "Accuracy" by question category. We find that PDFTriage tends to be stronger for categories like summarization, table reasoning, extraction, and figure questions which require multi-step reasoning across different parts of a document. Additionally, PDFTriage performs similarly to Page Retrieval and Chunk Retrieval on other more generalized reasoning tasks, such as text questions and classification.

6.3 PDFTriage requires fewer retrieved tokens to produce better answers

For the PDF document sample, the average token length of retrieved PDFTriage text is 1568 tokens (using the GPT-3.5 tokenizer). The average metadata length of textual inputs in document JSONs is 4,257 tokens (using the GPT-3.5 tokenizer).

While PDFTriage utilizes more tokens than Page Retrieval (3611 tokens on average) and Chunk Retrieval (3934 tokens on average), the tokens are retrieved from multiple sections of the document that are non-consecutive. Furthermore, the sections used in Page Retrieval and Chunk Retrieval are often insufficient for answering the question, as indicated by lower answer quality scores on average for "Overall Quality" and "Accuracy". However, simply concatenating all the document's text together would not ultimately replace PDFTriage due to both context window limits and the need to perform multi-hop reasoning for document QA tasks. PDFTriage helps overcome this issue through the multi-stage querying of the document, retrieving and adding context as needed for different document QA tasks.

Question	PDFTriage	Chunk Retrieval
Which file formats are listed under the “Create Using This Format” section?	Function used: `fetch_section()` The file formats listed under the "Create Using This Format" section are: CSV, JPG, LPT, PDF, PNG, PS, RTF, XLS, and XML.	The document does not provide information about the specific file formats listed under the “Create Using This Format” section.
Summarize the second paragraph in one long sentence.	Function used: `fetch_section()` The second paragraph explains the writer’s support for H.B. 5540, which aims to ban guns without serial numbers and regulate those sold in a form requiring assembly. The writer also supports the...	In her testimony, Michele Mudrick, Legislative Advocate for the Connecticut Conference United Church of Christ, expresses her support for H B 5540 and H B 5542, two acts concerning ghost guns, the permit application process, and...

Table 4: A comparison of `fetch_section()` being called successfully and unsuccessfully. Answers highlighted in red were considered incorrect. In the second example, both approaches are incorrect; the PDFTriage approach fetches the incorrect section, rather than just the first page, the chunk retrieval approach has no knowledge of document structure and paragraph order.

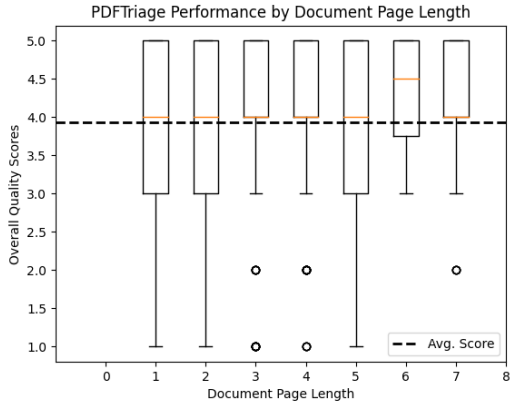

Figure 4: PDFTriage Performance compared to Document Page Length (uses "Overall Quality" scores)

6.4 PDFTriage performs consistently across document lengths

We also wanted to calculate the correlation between PDFTriage performance and the length of the document overall. Between the human-annotated PDFTriage answer score for "Overall Quality" and document length, we found a Pearson’s correlation coefficient of -0.015. This indicates that document length has a negligible effect on the efficacy of PDFTriage, strengthening the generalizability of our technique to both short and long documents.

The length of different document types seems to ultimately have no effect on overall performance. The ability of PDFTriage to query specific textual sections within the document prevents the need to ingest documents with excessively large contexts. It allows PDFTriage to connect disparate parts of a document for multi-page questions such as table reasoning, cross-page tasks, figure questions, and structure questions, prioritizing relevant context and minimizing irrelevant information. As a result, GPT-3 and other LLMs are better capable of handling the reduced context size and ultimately utilize less computational and financial resources for document QA tasks.

7 Future Work & Conclusions

In this work, we present PDFTriage, a novel question-answering technique specialized for document-oriented tasks. We compare our approach to existing techniques for question-answering, such as page retrieval and chunk retrieval, to demonstrate the strengths of our approach. We find that PDFTriage offers superior performance to existing approaches. PDFTriage also proves effective across various document lengths and contexts used for retrieval. We are considering the following directions for future work:

1. Developing multi-modal approaches that incorporate table and figure information into GPT-4 question-answering for documents.

2. Incorporate question type in PDFTriage approach to improve efficiency and efficacy of the approach.

References

Akari Asai, Timo Schick, Patrick Lewis, Xilun Chen, Gautier Izacard, Sebastian Riedel, Hannaneh Hajishirzi, and Wen-tau Yih. 2022. Task-aware retrieval with instructions. *arXiv preprint arXiv:2211.09260*.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011*.

Jiazhan Feng, Chongyang Tao, Xiubo Geng, Tao Shen, Can Xu, Guodong Long, Dongyan Zhao, and Daxin Jiang. 2023. Knowledge refinement via interaction between search engines and large language models. *arXiv preprint arXiv:2305.07402*.

Rudolph Flesch. 1948. A new readability yardstick. *Journal of applied psychology*, 32(3):221.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2022. Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496*.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. 2023. Reinforced self-training (rest) for language modeling.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466.

Jordy Landeghem, Rubén Tito, Łukasz Borchmann, Michał Pietruszka, Paweł Józiak, Rafał Powalski, Dawid Jurkiewicz, Mickaël Coustaty, Bertrand Ackaert, Ernest Valveny, et al. 2023. Document understanding dataset and evaluation (dude). *arXiv preprint arXiv:2305.08455*.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A benchmark for tool-augmented llms. *arXiv preprint arXiv:2304.08244*.

Yaobo Liang, Chenfei Wu, Ting Song, Wenshan Wu, Yan Xia, Yu Liu, Yang Ou, Shuai Lu, Lei Ji, Shaoguang Mao, et al. 2023. Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis. *arXiv preprint arXiv:2303.16434*.

Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. How to train your dragon: Diverse augmentation towards generalizable dense retrieval. *arXiv preprint arXiv:2302.07452*.

Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. 2021. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2200–2209.

Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. Augmented language models: a survey. *arXiv preprint arXiv:2302.07842*.

Aaron Parisi, Yao Zhao, and Noah Fiedel. 2022. Talm: Tool augmented language models. *arXiv preprint arXiv:2205.12255*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*.

Jayr Pereira, Robson Fidalgo, Roberto Lotufo, and Rodrigo Nogueira. 2023. Visconde: Multi-document qa with gpt-3 and neural reranking. In *European Conference on Information Retrieval*, pages 534–543. Springer.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355, Brussels, Belgium. Association for Computational Linguistics.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Zichun Yu, Chenyan Xiong, Shi Yu, and Zhiyuan Liu. 2023. Augmentation-adapted retriever improves generalization of language models as generic plug-in. *arXiv preprint arXiv:2305.17331*.

Ruochen Zhao, Hailin Chen, Weishi Wang, Fangkai Jiao, Xuan Long Do, Chengwei Qin, Bosheng Ding, Xiaobao Guo, Minzhi Li, Xingxuan Li, et al.

2023. Retrieving multimodal information for augmented generation: A survey. *arXiv preprint arXiv:2303.10868*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. Toolqa: A dataset for llm question answering with external tools. *arXiv preprint arXiv:2306.13304*.

A Appendix

A.1 Question Categories and Examples

In Table 6, and Table 7, we present descriptions as well as positive and negative examples for each question category. Each question category seeks to capture a different document question-answering task that is relevant across various professional fields.

A.2 Annotator Demographic Information

We used UpWork to recruit 12 English-speaking annotators to judge the answers generated by PDF-Triage and the baseline approaches. We paid all the annotators the same standard rate used for US annotators. Here is the demographic breakdown of annotators:

- 4 participants were located in India
- 2 participants were located in Pakistan
- 2 participants were located in South Africa
- 2 participants were located in Australia
- 2 participants were located in the Phillipines
- 2 participants were located in the United States.

Category	Positive Examples
Figure Questions	What is the main takeaway of Figure 4? What is the largest value in Figure 4? What kind of graph is used on page 5?
Text Questions	Is 2pm on Wednesday free? What evidence is used to support the author's conclusion in section #5?
Table Reasoning	Can you convert the minutes column in Table 2 to hours? What row has the maximum value of the "Accuracy" column?
Structure Questions	What is the main takeaway from section 5? What counterexamples are provided in paragraph 3, section #1?
Summarization	Can you provide a concise summary of section 2? Write a detailed summary about the main takeaways of the paper.
Extraction	Find all the council members mentioned in this document. What are the three central claims of the author? What are the main findings?
Rewrite	- Can you rewrite this in more modern language: "The thousand injuries of Fortunato I had borne as best I could. But when he ventured upon insult, I vowed revenge." - Can you simplify this: "In mice, immunoregulatory APCs express the dendritic cell (DC) marker CD11c, and one or more distinctive markers (CD8, B220, DX5)."
Outside Questions (Closed-book QA)	What other books were written by the novelist author? Besides the theory discussed in this document, what other scientific theories explain the given phenomena? Can you explain the term "mitochondria"?
Cross-page Tasks	Do the results in the conclusions support the claims in the abstract?
Classification	Is this document a scientific article? Is this document about a residential lease or a commercial lease?
Trick Question	A good trick question might: (a) be related to the document (b) refer to non-existent tables, figures, or sections (c) not have enough information to answer it (d) not be related to the document at all

Table 6: Positive Examples for Question Categories

Category	Negative Examples
Figure Questions	What is the main takeaway of the second graph. (missing reference to page or figure number)
Text Questions	What is the title of subsection #4? (too easy to answer)
Table Reasoning	What value is in the third column, fourth row? (too easy to answer)
Structure Questions	How many sections are there in the document? (too easy to answer) What is the title of the document? (too easy to answer)
Summarization	What is a summary of the document? (does not specify summary length) Write a short summary. (does not specify summary content)
Extraction	"How many times does the author mention the title character?" (not relevant question)
Rewrite	Remove all typos. (too broad, does not refer to specific text)
Outside Questions (Closed-book QA)	Questions that are unrelated to the document's content
Cross-page Tasks	Any task that is answerable in one place in the document, or not answerable at all.
Classification	Categories that are unrelated to the document.
Trick Question	

Table 7: Negative Examples for Question Categories

B Evaluation Details

B.1 Human Evaluation Interface

Question: What are the buttons show on the bottom of page 2 used for?

What type of question is this? (Select all that apply). Consider what the question is asking and what reasoning over the document content is required to arrive to an answer.

- ☐ Easy (is directly answerable in a single place)
- ☐ Medium (is directly answerable but spread across the document)
- ☐ Hard (requires generalization, summarization, e.g., “what is this about?”, “key points”, or “main takeaways”,)
- ☐ Hard (requires logic, math, comparison, counting, cause and effect)
- ☐ Hard (requires judgement, opinion, assessment)
- ☐ Hard (contains vague or unclear language)
- ☐ Asks for help using acrobat (e.g., fill a form, add a signature, etc)
- ☐ Has a reference to the layout of the document (Examples: “On page 6”, “In section 2”, “in paragraph 3”. Or asking how many pages, tables, paragraphs or sections are present)
- ☐ Unsure (explain)

If unsure for previous question, please explain here:

Figure 5: Annotation Question #1

Classify the given question into one of the following categories:

Question: What are the buttons show on the bottom of page 2 used for?

- ○ Figure Questions
- ○ Text Questions
- ○ Table Reasoning
- ○ Structure Questions
- ○ Summarization
- ○ Extraction
- ○ Rewrite
- ○ Outside Questions (Closed-Book QA)
- ○ Cross-page Tasks
- ○ Classification
- ○ Trick Question

Figure 6: Annotation Question #2

Consider the following question and provided document. Then, compare the following answers and rank answers from best to worst using their corresponding letters (best at the top).

Question: What are the buttons show on the bottom of page 2 used for?

A: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

B: The document provided does not mention any buttons on the bottom of page 2. Could you please provide more context or clarify your question?

C: The document does not mention any buttons on the bottom of page 2.

A

B

C

Figure 7: Annotation Question #3

Question: What are the buttons show on the bottom of page 2 used for?

Please rate the following answer for the following attribute:

Answer: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

Accuracy: The answer contains the correct information for answering the question

- ○ 1 - Strongly Disagree
- ○ 2 - Disagree
- ○ 3 - Neutral
- ○ 4 - Agree
- ○ 5 - Strongly Agree

Figure 8: Annotation Question #4

Question: What are the buttons show on the bottom of page 2 used for?

Please rate the following answer for the following attribute:

Answer: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

Informativeness: The answer contains enough information for the question.

- ○ 1 - Strongly Disagree
- ○ 2 - Disagree
- ○ 3 - Neutral
- ○ 4 - Agree
- ○ 5 - Strongly Agree

Figure 9: Annotation Question #5

Question: What are the buttons show on the bottom of page 2 used for?

Please rate the following answer for the following attribute:

Answer: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

Readability/Understandable: The answer is understandable and easy to read.

- ○ 1 - Strongly Disagree
- ○ 2 - Disagree
- ○ 3 - Neutral
- ○ 4 - Agree
- ○ 5 - Strongly Agree

Figure 10: Annotation Question #6

Question: What are the buttons show on the bottom of page 2 used for?

Please rate the following answer for the following attribute:

Answer: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

Clarity: The answer contains 'I don't know' or there is 'insufficient context' and there is NO answer provided.

- ○ 1 - Strongly Disagree
- ○ 2 - Disagree
- ○ 3 - Neutral
- ○ 4 - Agree
- ○ 5 - Strongly Agree

Figure 11: Annotation Question #7

Question: What are the buttons show on the bottom of page 2 used for?

Please rate the following answer for the following attribute:

Answer: The buttons shown at the bottom of page 2 are used for performing actions related to running and managing the tasks. The available buttons are:

1. "Run Task" - This button is used to run the selected task immediately.
2. "Save Changes" - This button is used to save any changes made to the task settings.
3. "Undo Changes" - This button is used to undo any changes made to the task settings and revert back to the last saved version.
4. "Help" - This button provides access to the online help documentation for more information on each task.
5. "Close" - This button is used to close the System Tasks window.

These buttons allow users to interact with the system tasks and perform tasks such as running, saving, undoing changes, accessing help documentation, and closing the System Tasks window.

Overall Quality: The overall quality of the answer.

- ○ 1 - Strongly Disagree
- ○ 2 - Disagree
- ○ 3 - Neutral
- ○ 4 - Agree
- ○ 5 - Strongly Agree

Figure 12: Annotation Question #8

B.2 GPT Evaluation and Discussion

For each question and document pair in our PDFTriage document sample, we gather the corresponding PDFTriage, Page Retrieval, and Chunks Retrieval answers for comparison. Next, for automatic evaluation, we use the *gpt-3.5-turbo* model since we used the same model for our PDFTriage system and comparative baselines. We query the model using the following system prompt:

> Give a score (1-5) for how well the question was answered. Only provide the numerical rating. Do not give any explanation for your rating.
>
> Question: `<question here>`

Answer: <answer here>

Between our GPT-4 evaluation scores and the "Overall Quality" score of the human annotations, we calculated a Cohen's kappa score of 0.067 and a Pearson's correlation coefficient of 0.19 across the entire dataset. Both these metrics indicate a negligible alignment between the GPT-4 evaluation scores and the human annotations.

Therefore, we believe the automated GPT-4 evaluation requires further instructions or fine-tuning to better align with human preferences for document question-answering tasks. Recent work has taken steps towards improving automated LLM evaluation alignment with human preferences (Zheng et al., 2023; Gulcehre et al., 2023). For future research, it would be worth considering how we can leverage few-shot prompt-tuning to better align generative LLMs with human preferences in evaluation tasks.

B.3 Performance vs. Context Window Trade-off

To better understand the connection between PDFTriage performance and the length of the context window of the text retrieved from the document, we calculated the correlation between the human annotators' scores for PDFTriage answers and the length of the context retrieved from the document metadata. We found that the Pearson's correlation coefficient is 0.062, indicating a negligible connection between the retrieved context of PDFTriage and its overall efficacy.

Interestingly, it seems like longer context length does not improve PDFTriage performance, according to the human annotations. PDFTriage instead needs to query the *precise* information needed for answering different document QA questions, particularly those like cross-page tasks and structure questions which require multiple stages of querying. This suggests that full-concatenation of the document text wouldn't necessarily improve document QA performance since additional text does not correlate with improved accuracy or overall quality scores for the answers.

B.4 Evaluation Breakdown by Question Category

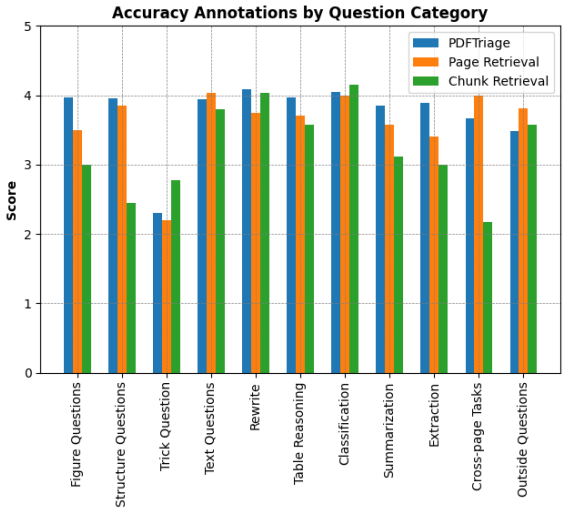

Figure 13: Accuracy Annotation Scores by Question Category

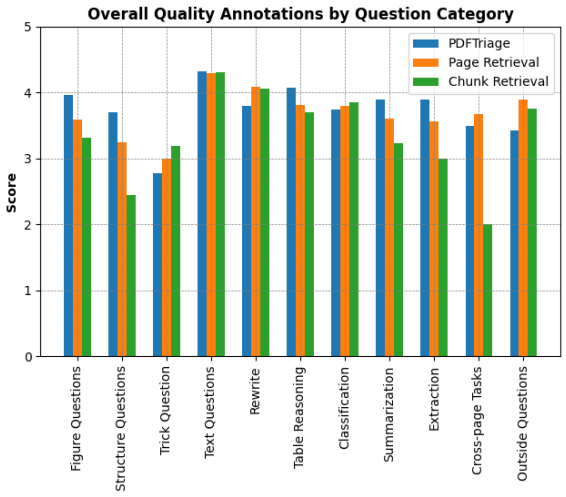

Figure 14: Overall Quality Annotation Scores by Question Category

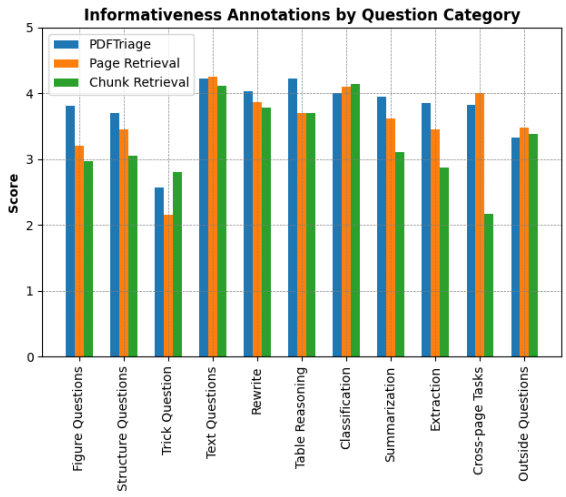

Figure 15: Informativeness Annotation Scores by Question Category

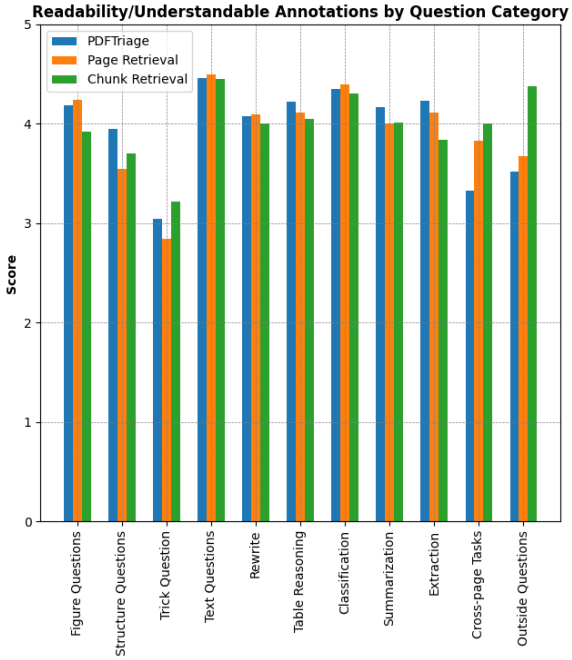

Figure 17: Readability Annotation Scores by Question Category

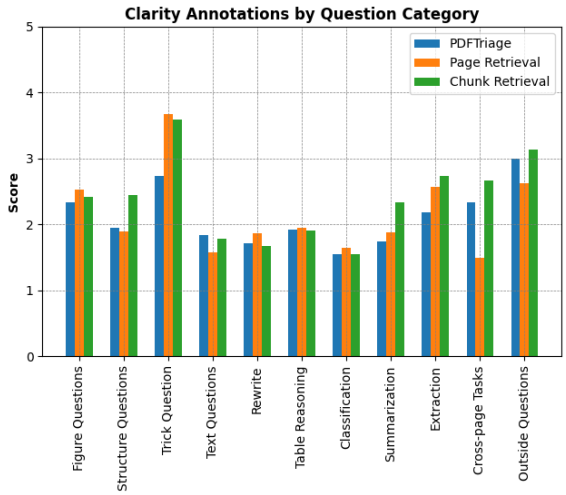

Figure 16: Clarity Annotation Scores by Question Category